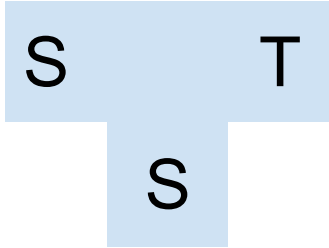


Lecture 14

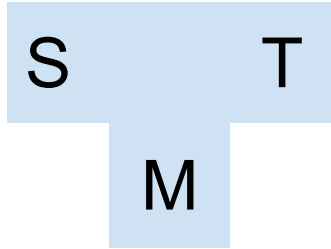
Tokenization, Trefoil v1

Bootstrapping Compilers

- S compiler written in S
- No way to run the compiler!

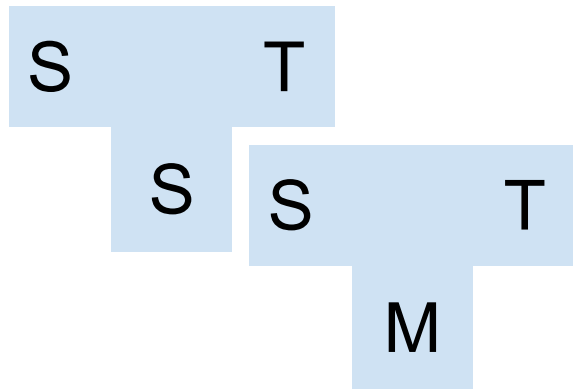


Bootstrapping Compilers



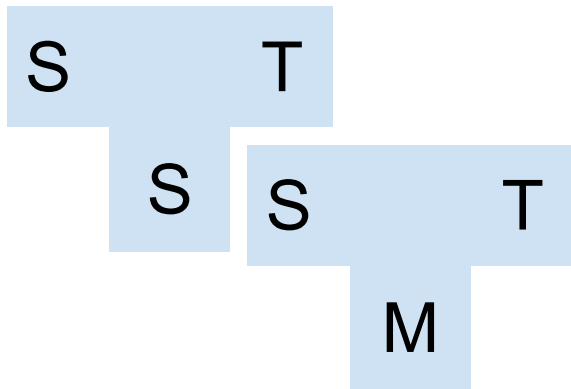
- S compiler written in M
- Need to have some way to compile/run M code
- Now we have a way to compile S code

Bootstrapping Compilers



- Use that compiler to compile the S compiler written in S
- Now we have a way to compile S code that doesn't rely on being able to run M
 - Good for portability to different environments
 - Good for maintainability

Bootstrapping Compilers



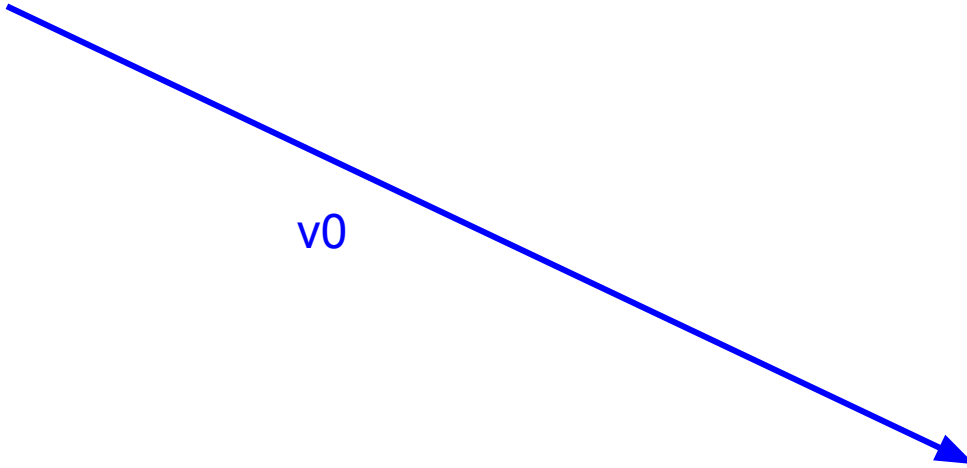
- Use that compiler to compile the S compiler written in S
- Now we have a way to compile S code that doesn't rely on being able to run M
 - Good for portability to different environments
 - Good for maintainability
- To add a language feature to S:
 - First add support for the feature in the compiler (without using the new feature)
 - Compile the compiler
 - Now update the compiler to use the new feature (if desired)

Anatomy of an Interpreter

Input String

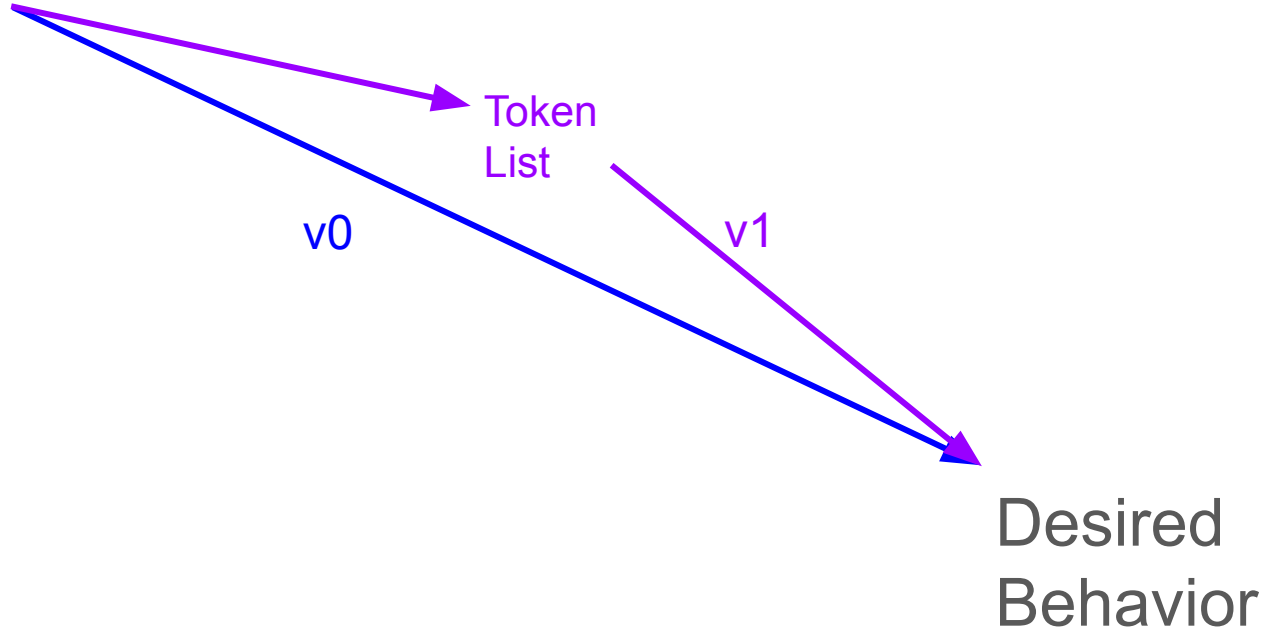
v0

Desired
Behavior



Anatomy of an Interpreter

Input String



Tokens (In General)

Indivisible piece of input program

- Integer literal (ex. 1234 or -678)
- Variable name (ex. foo or my_var1)
- Punctuation (ex. (or +)
- String literal (ex. “something something” or “multiline \n string”)

Tokens (In Trefoil v1)

Indivisible piece of input program

- Integer literals
- Operators: $+$ $-$ $*$ $.$

Trefoil v1 : Tokenization

`string -> token list`

- space delimited, so use OCaml's built-in split operation
- Now have a list of strings
- Look at each string, try to match it against one of the possible tokens
 - For integer literals, use OCaml's string to int conversion
 - If nothing matches, report an error

Group Work

1. Write down an explanation of the semantics of Trefoil v1
2. Write 2 more Trefoil v1 programs along with their expected output that test your hypothesis

Program	Behavior
1 2 + .	prints 3
1 2 + 3 * .	prints 9
1 .	prints 1
1 2 .	prints 2
1 2 . .	prints 2 and then prints 1
1 2 +	doesn't print anything
3 1 2 + * .	prints 9
3 1 2 + * 1 - .	prints 8
10 1 + .	prints 11
1 2 + 1 2 + * .	prints 9

Group Work

Program	Behavior
1 2 + .	prints 3
1 2 + 3 * .	prints 9
1 .	prints 1
1 2 .	prints 2
1 2 . .	prints 2 and then prints 1
1 2 +	doesn't print anything
3 1 2 + * .	prints 9
3 1 2 + * 1 - .	prints 8
10 1 + .	prints 11
1 2 + 1 2 + * .	prints 9

Trefoil v1 : Interpretation

token list -> desired behavior

- Initialize an empty stack
- Look at the tokens in the list from left to right
 - If the next token is an integer literal, push it on the stack
 - Implement the operator tokens by popping args and pushing results (one case per operator)
 - Also handle error cases

Parsing

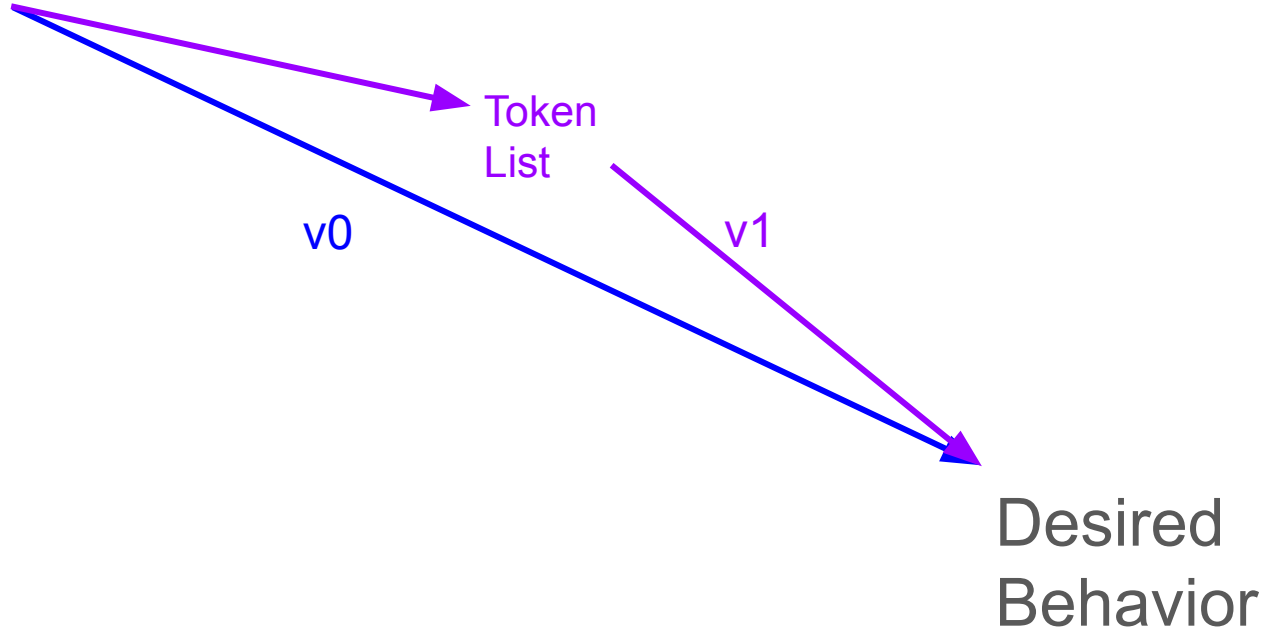
Infix Arithmetic Expressions

An expression is

- An int literal
- Add operator : $e1 + e2$ (where $e1$ and $e2$ are expressions)
- Sub operator : $e1 - e2$
- Mul operator : $e1 * e2$

Anatomy of an Interpreter

Input String



Anatomy of an Interpreter

Input String

Token
List

Token
List

```
type expr =  
  | Int of int  
  | Add of expr * expr  
  | Sub of expr * expr  
  | Mul of expr * expr
```

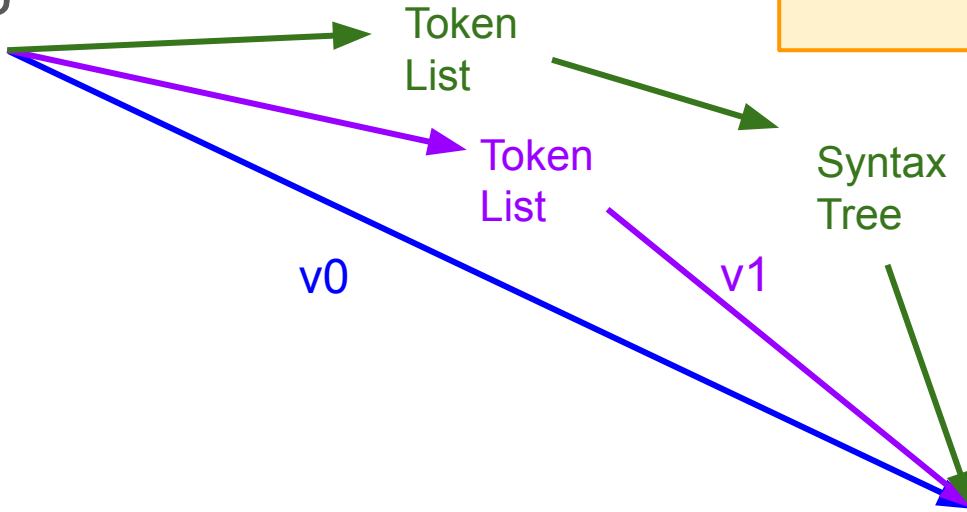
Syntax
Tree

v0

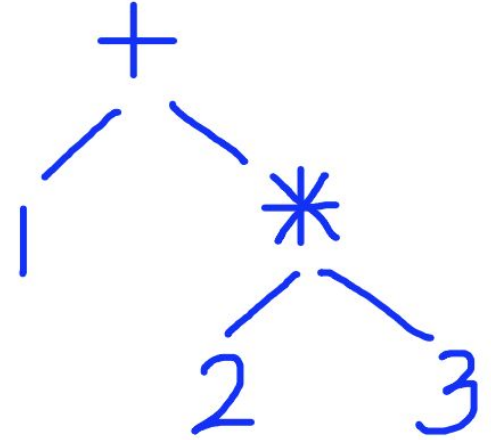
v1

v2

Desired
Behavior

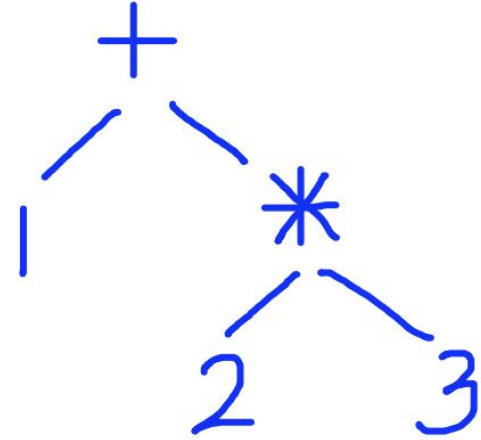


Infix Arithmetic Interpreter : $AST \rightarrow integer$



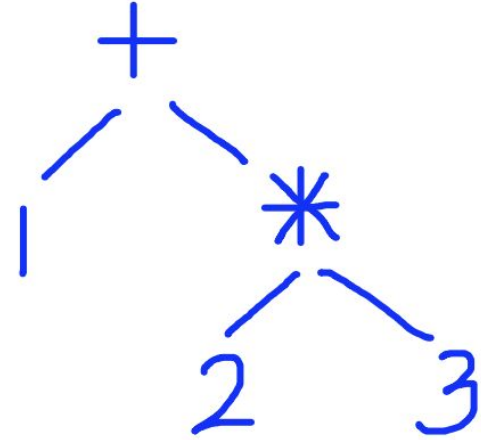
Infix Arithmetic Interpreter : $AST \rightarrow integer$

- If input is int literal, return that
- If input is an operator:
 - Recursively interpret children
 - Then apply operator to results



Infix Arithmetic Interpreter : $AST \rightarrow integer$

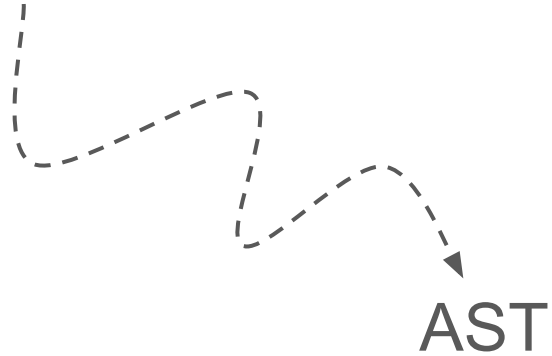
- If input is int literal, return that
- If input is an operator:
 - Recursively interpret children
 - Then apply operator to results



THE RECIPE

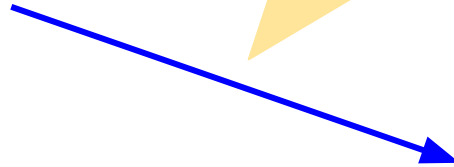
Infix Arithmetic Interpreter : $AST \rightarrow integer$

Input String



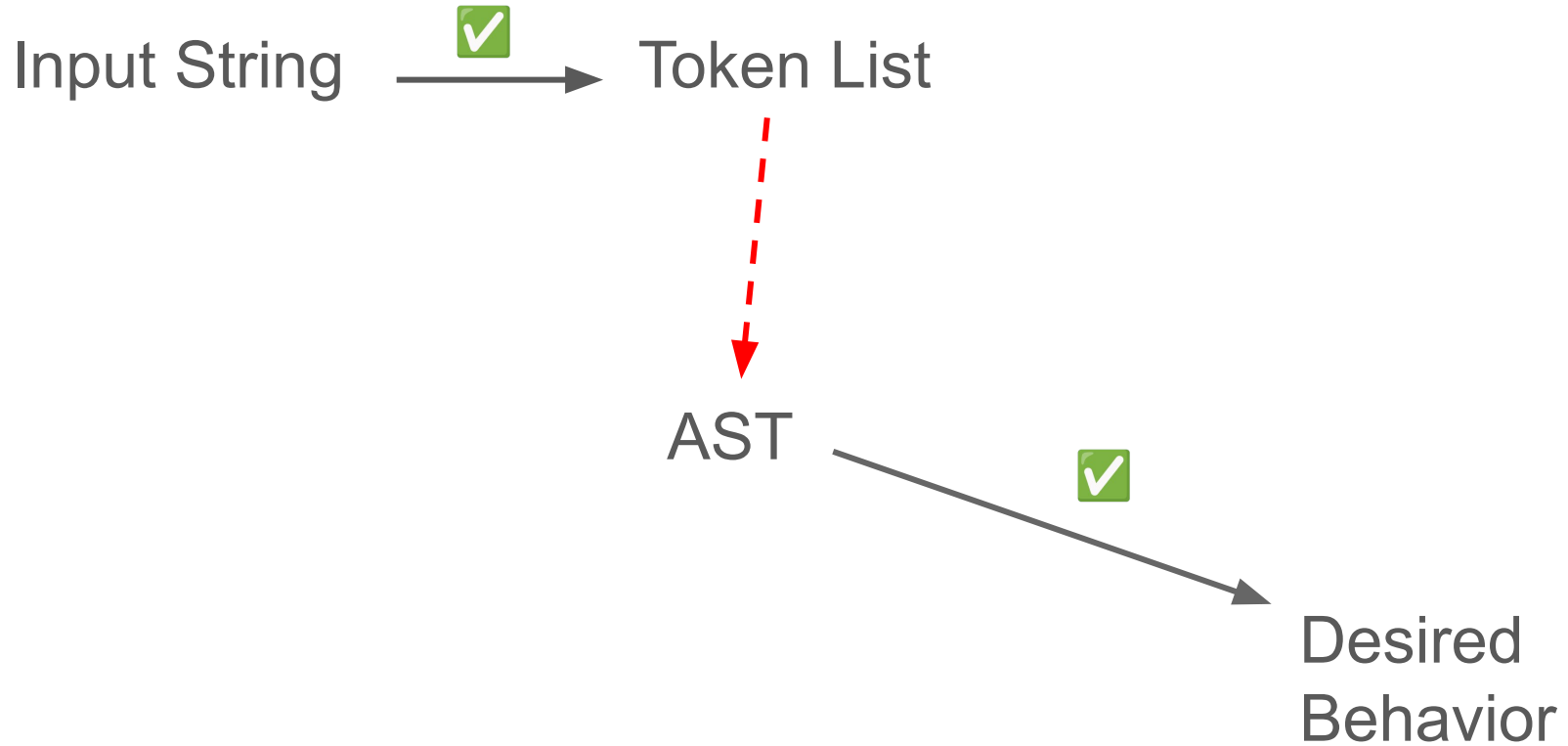
AST

To the laboratory!



Desired
Behavior

Parsing



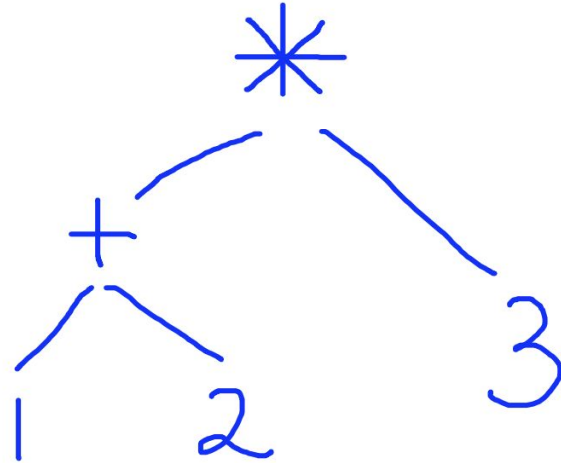
Tokenization + Parsing

1 2 + 3 *

Token List:

- Int (1)
- Int (2)
- Plus
- Int (3)
- Times

Abstract Syntax Tree



Tokenization + Parsing

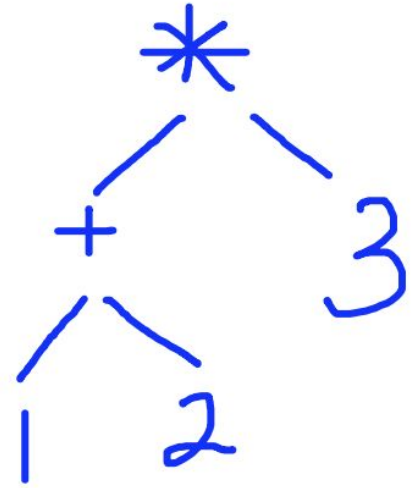
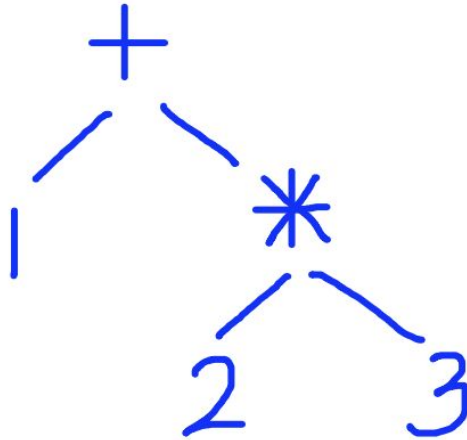
1 + 2 * 3

Abstract Syntax Trees

Which one?

Token List:

- Int (1)
- Plus
- Int (2)
- Times
- Int (3)



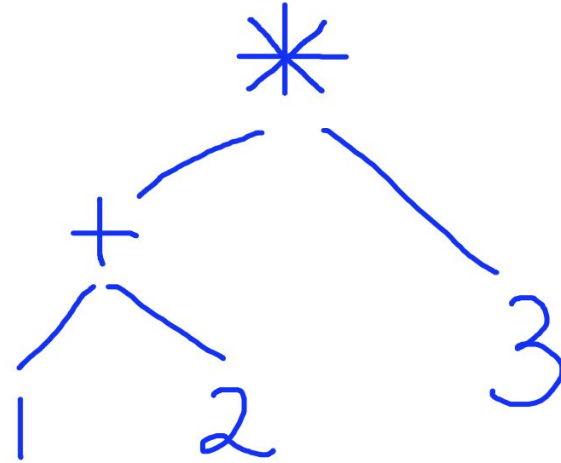
Tokenization + Parsing

(1 + 2) * 3

Token List:

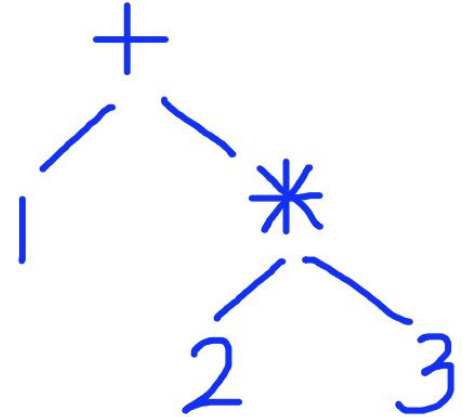
- LParen
- Int (1)
- Plus
- Int (2)
- RParen
- Times
- Int (3)

Abstract Syntax Tree



Parenthesized Syntax Trees (S-expressions)

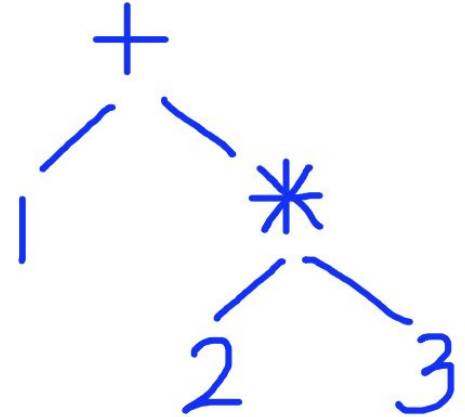
Hard to parse: 1 + 2 * 3



Parenthesized Syntax Trees (S-expressions)

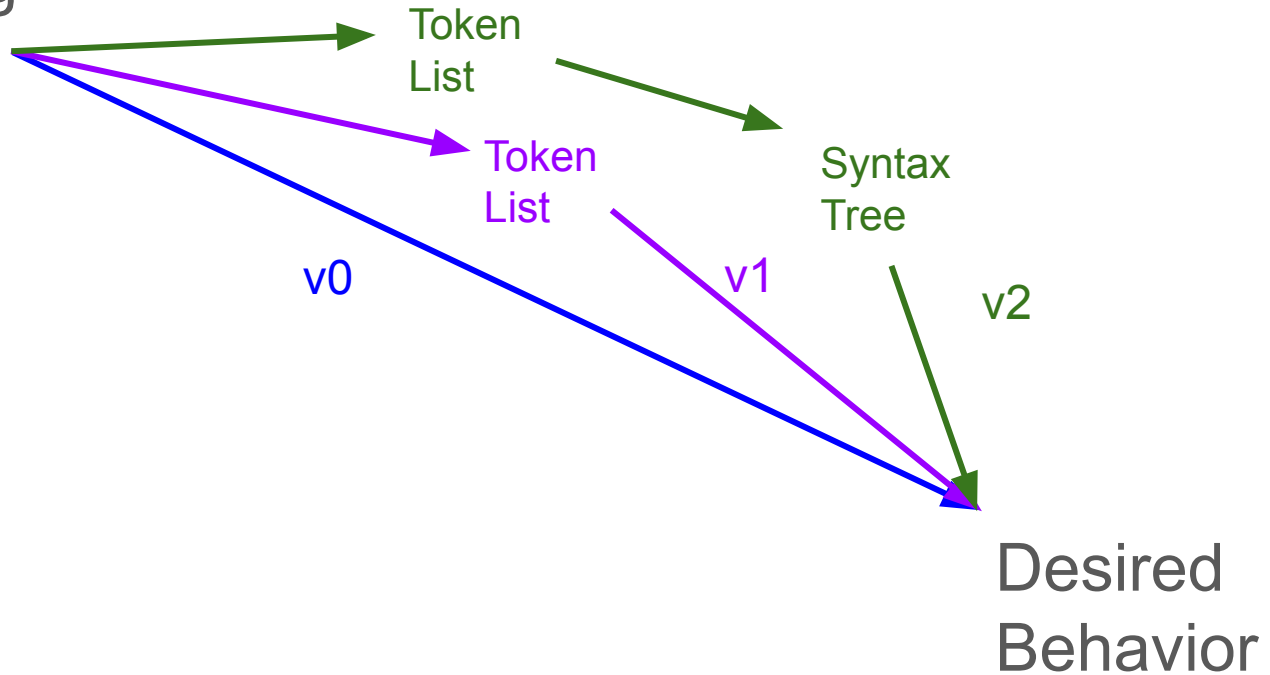
Hard to parse: $1 + 2 * 3$

Easy to parse: $(+ 1 (* 2 3))$



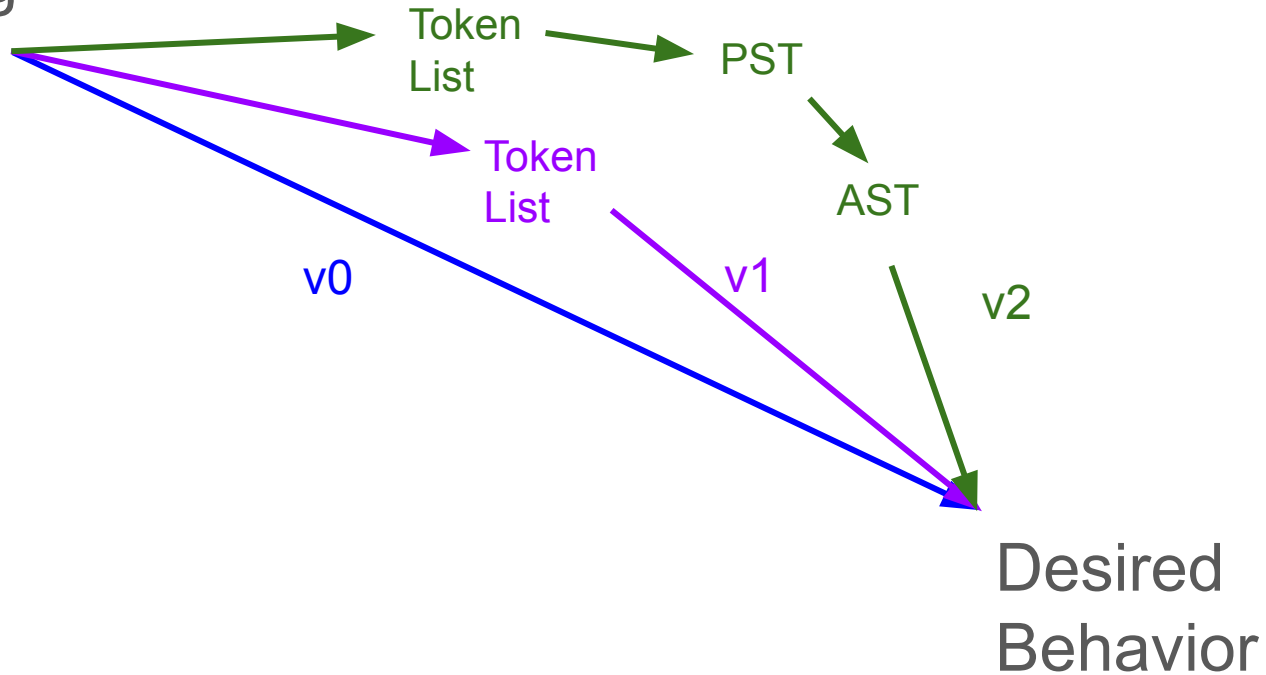
Anatomy of an Interpreter

Input String



Anatomy of an Interpreter

Input String



Parenthesized Syntax Trees

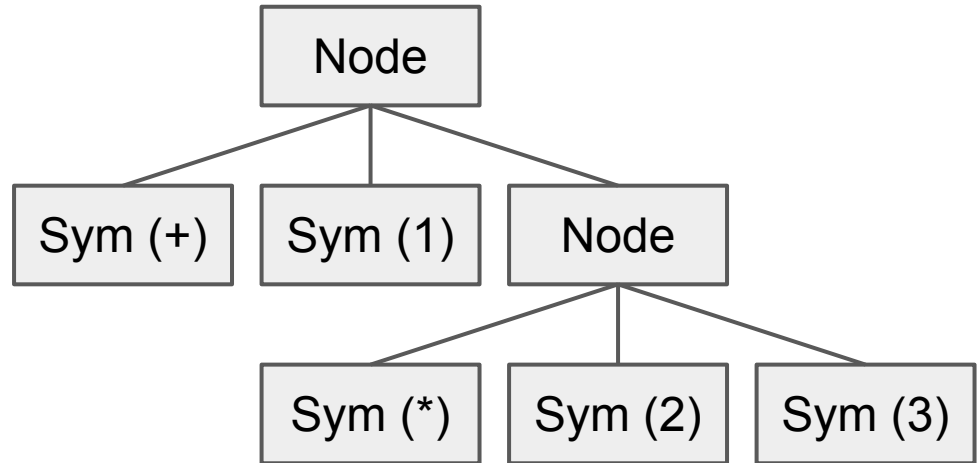
- A symbol (String of non-whitespace, non-paren characters)
- A node, which has a list of PST children

```
type pst =  
  | Symb of string  
  | Node of pst list
```

Parenthesized Syntax Trees

- A symbol (String of non-whitespace, non-paren characters)
- A node, which has a list of PST children

(+ 1 (* 2 3))



Group Work

For the Trefoil v2 program $(+ (+ 1 7) (* (- 3 5) 6))$, write:

- PST (**pst**)
- AST (**expr**)
- Desired Behavior

```
type expr =  
  | Int of int  
  | Add of expr * expr  
  | Sub of expr * expr  
  | Mul of expr * expr
```

```
type pst =  
  | Symb of string  
  | Node of pst list
```